

Atelier 5:

Application Microservices

Node.js + MongoDB Atlas

1. Installation des packages (mongoose):

```
houdaifa@houdaifa:~/everything/node/ateliers/atelier5/books$ npm install mongoose
added 17 packages, and audited 83 packages in 9s
23 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
houdaifa@houdaifa:~/everything/node/ateliers/atelier5/books$
```

2. Book model:

```
atelier5 > books > api > JS Book.js > ...
1  const mongoose = require("mongoose");
2  mongoose.model("Book", {
3    // Title, author, numberPages, publisher
4    title: {
5      type: String,
6      require: true
7    },
8    author: {
9      type: String,
10     require: true
11   },
12   numberPages: {
13     type: Number,
14     require: false
15   },
16   publisher: {
17     type: String,
18     require: false
19   }
20 }
21 });
22
```

3. Require Books model dans books.js:

```
4
5 // Import Book model
6 require("./Book");
7
```

4. Connections to mongoDB:

```
// Connect to MongoDB
const uri = "mongodb+srv://houdaifazaidi04_db_user:xmlWwoqrHcc1SakEo@cluster0.pufzrga.mongodb.net/?appName=Cluster0";

mongoose.connect(uri)
  .then(() => console.log("Connected to MongoDB !!!"))
  .catch(err => console.error("Could not connect to MongoDB", err));

// Route 1 - Home
app.get('/', (req, res) => {
  res.send("Welcome to customers service !!!");
});
```

5. Post (ajouter un livre):

```
// Get Book model
const Book = mongoose.model("Book");

// Add new book
app.post("/book", (req, res) => {

  const newBook = {
    title: req.body.title,
    author: req.body.author,
    publisher: req.body.publisher
  };

  const book = new Book(newBook);

  book.save()
    .then(() => {
      console.log("Book created");
      res.json({ message: "New book added" });
    })
    .catch(err => {
      res.status(500).json({ error: "Error creating book" });
    });
});
```

6. Get (tous les livres):

```
// Get all books
app.get("/books", (req, res) => {

  Book.find()
    .then(books => {
      res.json({ books: books });
    })
    .catch(err => {
      res.status(500).json({ error: "Error fetching books" });
    });

});

// Get book by id
```

7. Get (livre par id):

```
// Get book by id
app.get("/books/:id", (req, res) => {

  Book.findById(req.params.id)
    .then(book => {
      if (book) {
        res.json({ book: book });
      } else {
        res.status(404).json({ message: "Book not found" });
      }
    })
    .catch(err => {
      res.status(500).json({ error: "Invalid ID" });
    });

});
```

8. Delete (supprimer un livre):

```
// Delete book by id
app.delete("/books/:id", (req, res) => {

  Book.findByIdAndDelete(req.params.id)
    .then(() => {
      res.json({ message: "Book deleted" });
    })
    .catch(err => {
      res.status(500).json({ error: "Error deleting book" });
    });
});
```

9. Tests:

a. Ajouter un livre:

The screenshot displays a REST client interface with the following details:

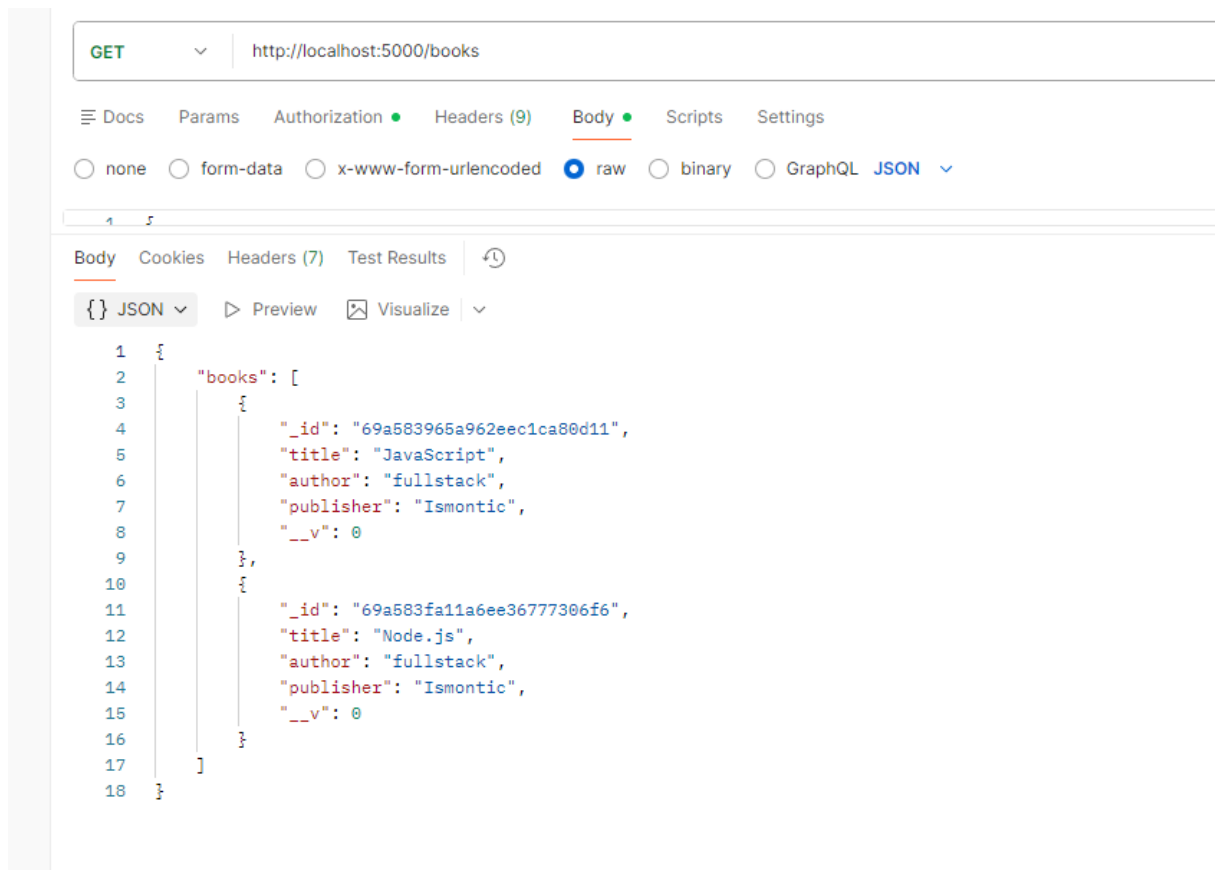
- Method:** POST
- URL:** http://localhost:5000/book
- Body Type:** raw
- Request Body (JSON):**

```
{
  "title": "JavaScript",
  "author": "fullstack",
  "publisher": "Ismontic"
}
```
- Response Body (JSON):**

```
{ "message": "a new book added !!!" }
```
- Bottom Panel:** Shows a database cluster named 'Cluster0' with a 'books' collection. The document details are:

```
{
  _id: ObjectId('69a583965a962eec1ca80d11'),
  title: "JavaScript",
  author: "fullstack",
  publisher: "Ismontic",
  __v: 0
}
```

b. Recuperer Tous les livres:



GET ▼ http://localhost:5000/books

Docs Params Authorization Headers (9) **Body** Scripts Settings

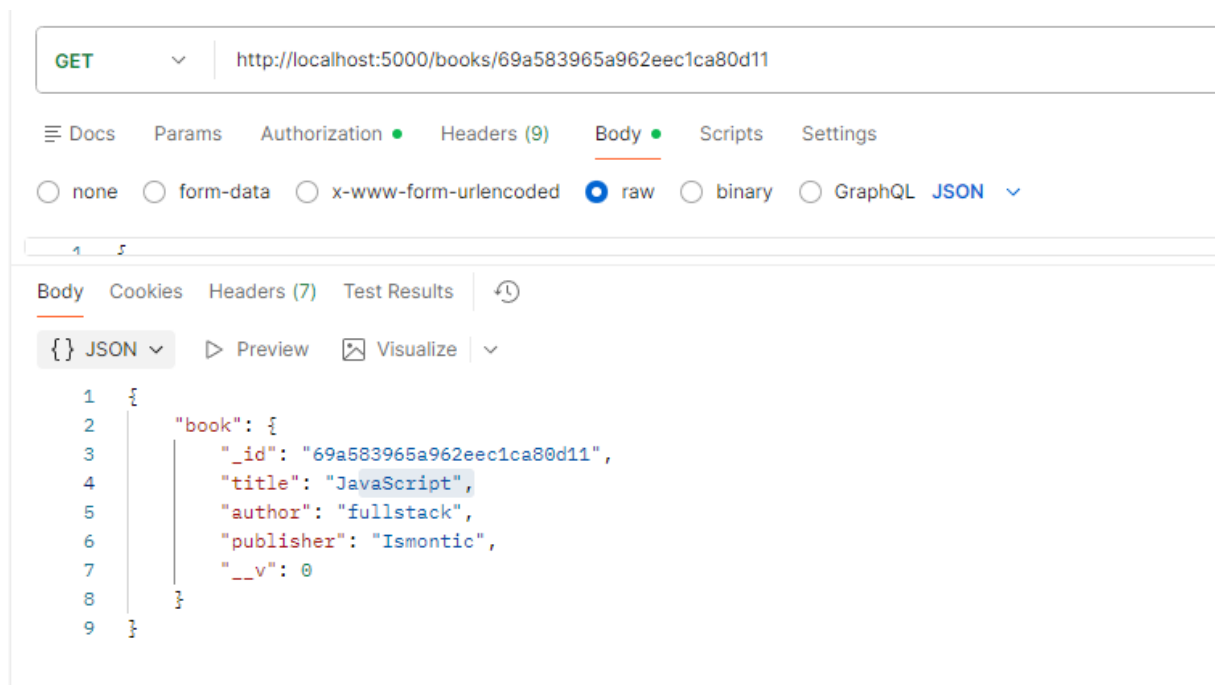
☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

Body Cookies Headers (7) Test Results ↺

{} JSON ▼ ▶ Preview 🖼 Visualize ▼

```
1 {
2   "books": [
3     {
4       "_id": "69a583965a962eec1ca80d11",
5       "title": "JavaScript",
6       "author": "fullstack",
7       "publisher": "Ismontic",
8       "__v": 0
9     },
10    {
11      "_id": "69a583fa11a6ee36777306f6",
12      "title": "Node.js",
13      "author": "fullstack",
14      "publisher": "Ismontic",
15      "__v": 0
16    }
17  ]
18 }
```

c. Recuperer un livre par id :



GET ▼ http://localhost:5000/books/69a583965a962eec1ca80d11

Docs Params Authorization Headers (9) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

Body Cookies Headers (7) Test Results ↺

{} JSON ▼ ▶ Preview 🖼 Visualize ▼

```
1 {
2   "book": {
3     "_id": "69a583965a962eec1ca80d11",
4     "title": "JavaScript",
5     "author": "fullstack",
6     "publisher": "Ismontic",
7     "__v": 0
8   }
9 }
```

d. Supprimer un livre :

The screenshot shows a REST client interface. At the top, a dropdown menu is set to 'DELETE' and the URL is 'http://localhost:5000/books/69a583965a962eec1ca80d11'. Below this, there are tabs for 'Docs', 'Params', 'Authorization', 'Headers (9)', 'Body', 'Scripts', and 'Settings'. The 'Body' tab is selected, and the content type is set to 'raw'. The request body is a JSON object:

```
1 {  
2   "title": "Node.js",  
3   "author": "fullstack",  
4   "publisher": "Ismontic"  
5 }
```

Below the request, there are tabs for 'Body', 'Cookies', 'Headers (7)', 'Test Results', and a refresh icon. The 'Body' tab is selected, and the content type is set to 'JSON'. The response body is a JSON object:

```
1 {  
2   "message": "Book deleted"  
3 }
```

10. Customer model :

```
atelier5 > customers > api > JS Customer.js > ...  
1  const mongoose = require("mongoose");  
2  
3  mongoose.model("Customer", {  
4    // name - email  
5    name: {  
6      type: String,  
7      require: true  
8    },  
9    email: {  
10     type: String,  
11     require: true  
12   }  
13  });  
14  
15
```

11. Post (ajouter) :

```
// Route 2 – Insert new customer
app.post("/customer", async (req, res) => {

  try {
    const newCustomer = new Customer({
      name: req.body.name,
      email: req.body.email
    });

    await newCustomer.save();

    console.log("New customer created!");
    res.json({ message: "A new customer added !!!" });

  } catch (err) {
    res.status(500).json({ error: "Error creating customer" });
  }

});
```

12. Get (par id) :

```
// Route 4 – Get customer by ID
app.get('/customers/:id', async (req, res) => {

  try {
    const customer = await Customer.findById(req.params.id);

    if (!customer) {
      return res.status(404).json({ message: "Customer not found" });
    }

    res.json({ customer });

  } catch (err) {
    res.status(500).json({ error: "Invalid ID" });
  }

});
```

13. Tests :

a. Ajouter un client :

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:6000/customer
- Body Tab:** Selected, showing a JSON object:

```
1 {
2   "name": "houdaifa",
3   "email": "houdaifa@email.com"
4 }
```
- Response Tab:** Selected, showing a JSON object:

```
1 {
2   "message": "New customer added"
3 }
```

```
_id: ObjectId('69a58c9bdfde24fb51b72880')
name : "houdaifa"
email : "houdaifa@email.com"
__v : 0
```

```
_id: ObjectId('69a58cabdfde24fb51b72882')
name : "Ali"
email : "ali@email.com"
__v : 0
```

b. Recuperer tous :

GET <http://localhost:6000/customers>

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ☐

```
1 {
2   "name": "Ali",
3   "email": "ali@email.coom"
4 }
```

Body Cookies Headers (7) Test Results

{ } JSON ☐ Preview Visualize ☐

```
1 {
2   "customers": [
3     {
4       "_id": "69a58c9bdfde24fb51b72880",
5       "name": "houdaifa",
6       "email": "houdaifa@email.coom",
7       "__v": 0
8     },
9     {
10      "_id": "69a58cabdfde24fb51b72882",
11      "name": "Ali",
12      "email": "ali@email.coom",
13      "__v": 0
14    }
15  ]
16 }
```

c. Supprimer un client :

DELETE <http://localhost:6000/customers/69a58cabdfde24fb51b72882>

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ☐

```
1 {
2   "name": "Ali",
3   "email": "ali@email.coom"
4 }
```

Body Cookies Headers (7) Test Results

{ } JSON ☐ Preview Visualize ☐

```
1 {
2   "message": "Customer deleted"
3 }
```

14. Post Order (ajouter un order) :

```
2 // Add new order
3 app.post("/order", (req, res) => {
4
5   const newOrder = {
6     CustomerID: new ObjectId(req.body.CustomerID),
7     BookID: new ObjectId(req.body.BookID),
8     initialDate: req.body.initialDate,
9     deliveryDate: req.body.deliveryDate
10  };
11
12  const order = new Order(newOrder);
13
14  order.save()
15    .then(() => {
16      console.log("Order created");
17      res.json({ message: "New order added" });
18    })
19    .catch(err => {
20      res.status(500).json({ error: "Error creating order" });
21    });
22
23 });
```

15. Get (order by Id) :

```
app.get("/order/:id", (req, res) => {  
  Order.findById(req.params.id)  
    .then(order => {  
      axios.get("http://localhost:5555/customers/" + order.CustomerID)  
        .then((response) => {  
          let orderObject = {  
            customerName: response.data.customer.name,  
            bookTitle: ""  
          };  
          axios.get("http://localhost:4545/books/" + order.BookID)  
            .then((response) => {  
              orderObject.bookTitle = response.data.book.title;  
              res.json({ order: orderObject });  
            })  
            .catch(err => {  
              res.status(500).json({ error: "Error fetching book" });  
            });  
        })  
        .catch(err => {  
          res.status(500).json({ error: "Error fetching customer" });  
        });  
      })  
      .catch(err => {  
        res.status(500).json({ error: "Invalid ID" });  
      });  
    });  
});
```

16. Tests :

a. Ajouter un order :

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3333/order
- Body Type:** raw (selected)
- Request Body (JSON):**

```
1 {  
2   "CustomerID": "69a58c9bdfde24fb51b72880",  
3   "BookID": "69a583fa11a6ee36777306f6",  
4   "initialDate": "2024-03-01",  
5   "deliveryDate": "2024-03-15"  
6 }
```
- Response Body (JSON):**

```
1 {  
2   "message": "New order added"  
3 }
```

b. Order by id :

The screenshot displays a REST client interface. At the top, a GET request is configured for the URL `http://localhost:4444/order/69a59119eef4b22ba672c20a`. The 'Body' tab is selected, showing a raw JSON payload. Below the request, the 'Response' section is visible, showing the JSON response received from the server.

Request:

```
1 {
2   "CustomerID": "69a58c9bdfde24fb51b72880",
3   "BookID": "69a583fa11a6ee36777306f6",
4   "initialDate": "2024-03-01",
5   "deliveryDate": "2024-03-15"
6 }
```

Response:

```
1 {
2   "order": {
3     "customerName": "houdaifa",
4     "bookTitle": "Node.js"
5   }
6 }
```